

Algoritmos esenciales con arreglos en C++

Inserción ordenada · Intercambio · Selección · Eliminación · Búsqueda binaria

Objetivo Comprender la estructura del algoritmo y luego adaptarlo al problema.	Idea clave Si existen arreglos paralelos, todos deben moverse o intercambiarse juntos.	Práctica Cambia el criterio, el sentido del orden y la condición de eliminación.
--	--	--

01 Inserción ordenada

Mantiene la misma estructura enviada: empieza desde la última posición ocupada, desplaza los datos mientras corresponda y finalmente inserta código y precio.

insertar_ordenado.cpp

```
1 void insertarOrdenado(int codigo, double precio,
2                       int *arrCodigo, double *arrPrecio,
3                       int &cantidad) {
4     int posInsercion = cantidad - 1;
5     while (posInsercion >= 0 and
6           arrPrecio[posInsercion] < precio) {
7         arrCodigo[posInsercion + 1] = arrCodigo[posInsercion];
8         arrPrecio[posInsercion + 1] = arrPrecio[posInsercion];
9         posInsercion--;
10    }
11
12    arrCodigo[posInsercion + 1] = codigo;
13    arrPrecio[posInsercion + 1] = precio;
14    cantidad++;
15 }
```

? **RETO:** agrega un segundo criterio cuando los precios sean iguales.

02 Ordenación por intercambio

Mantiene los dos ciclos anidados de la captura. El criterio se evalúa directamente en el if y, si se cumple, se intercambian todos los arreglos paralelos.

ordenacion_intercambio.cpp

```
1 void intercambiar(int &a, int &b) {
2     int auxiliar = a;
3     a = b;
4     b = auxiliar;
5 }
6
7 void intercambiar(double &a, double &b) {
8     double auxiliar = a;
9     a = b;
10    b = auxiliar;
11 }
12
13 void ordenarIntercambio(int *arrCodigo, int *arrCantidad,
14                        double *arrPrecio, int cantidad) {
15     for (int i = 0; i < cantidad - 1; i++) {
16         for (int j = i + 1; j < cantidad; j++) {
17             if (arrCantidad[i] < arrCantidad[j]) {
18                 intercambiar(arrCodigo[i], arrCodigo[j]);
19                 intercambiar(arrCantidad[i], arrCantidad[j]);
20                 intercambiar(arrPrecio[i], arrPrecio[j]);
21             }
22         }
23     }
24 }
```



RETO: cambia el signo del if o agrega más criterios de ordenación.

03 Ordenación por selección

Guarda la posición del menor, recorre el resto del arreglo y realiza un único intercambio al final de cada pasada.

ordenacion_seleccion.cpp

```
1  ///? Ordenamiento POR SELECCION
2  ///? [Eficiente pero al inicio puede resultar confuso]
3  ///? Encuentra POSICIONES; no intercambia en cada comparacion.
4  void selection_sort(double *arrNota, int n) {
5      for (int i = 0; i < n - 1; i++) {
6          int posMenor = i; // Se inicializa por fines pedagogicos.
7          ///? posMenor: ascendente - posMayor: descendente
8
9          for (int j = i + 1; j < n; j++) {
10             if (arrNota[j] < arrNota[posMenor]) {
11                 posMenor = j;
12             }
13         }
14
15         if (posMenor != i) {
16             // posMenor contiene la posicion encontrada.
17             intercambiar(arrNota[i], arrNota[posMenor]);
18         }
19     }
20 }
```

i CLAVE: se comparan muchas posiciones, pero solo se intercambia una vez por pasada.

04 Ordenación tipo burbuja

Compara elementos adyacentes. En cada pasada, el mayor de la zona pendiente avanza hasta el extremo derecho.

ordenacion_burbuja.cpp

```
1  /// Ordenamiento TIPO BURBUJA (DUALES)
2  /// Compara pares adyacentes hasta el limite de valores.
3  void bubble_sort(int *arr, int num) {
4      for (int i = 0; i < num - 1; i++) {
5          for (int j = 0; j < num - 1 - i; j++) {
6              if (arr[j] > arr[j + 1]) {
7                  int aux = arr[j];
8                  arr[j] = arr[j + 1];
9                  arr[j + 1] = aux;
10             }
11         }
12     }
13 }
```

05 Eliminar datos del arreglo

Conserva el contador j de la captura: copia hacia adelante solo los registros que deben permanecer y al final actualiza la cantidad lógica.

eliminar_datos.cpp

```
1  void eliminarDatos(int *arrCodigo, int *arrCantidad,
2                    double *arrPrecio, int &cantidad) {
3      int j = 0;
4
5      for (int i = 0; i < cantidad; i++) {
6          if (arrCantidad[i] != 0) {
7              arrCodigo[j] = arrCodigo[i];
8              arrCantidad[j] = arrCantidad[i];
9              arrPrecio[j] = arrPrecio[i];
10             j++;
11         }
12     }
13
14     cantidad = j;
15 }
```

06

Búsqueda binaria

Mantiene el `while(true)`, los límites inferior y superior, el punto medio y la búsqueda ascendente mostrada en la captura.

busqueda_binaria.cpp

```
1  const int NO_ENCONTRADO = -1;
2
3  int busquedaBinaria(int *arrDato, int dato,
4                      int cantidad) {
5      int limiteInferior = 0;
6      int limiteSuperior = cantidad - 1;
7      int puntoMedio;
8
9      while (true) {
10         if (limiteInferior > limiteSuperior)
11             return NO_ENCONTRADO;
12         puntoMedio = (limiteInferior + limiteSuperior) / 2;
13         if (dato == arrDato[puntoMedio]) return puntoMedio;
14         if (dato > arrDato[puntoMedio])
15             limiteInferior = puntoMedio + 1;
16         else
17             limiteSuperior = puntoMedio - 1;
18     }
19 }
```

Variaciones que podrían pedirte

Inserción ordenada	Pueden pedir más de un criterio de ordenación o una condición extra en el <code>while</code> de inserción.
Intercambio	Pueden solicitar varios criterios y el orden ascendente o descendente. Recuerda intercambiar todos los arreglos paralelos.
Búsqueda binaria	Se puede buscar en un arreglo ordenado ascendente o descendente; cambia la decisión de qué mitad conservar.



LOS ALGORITMOS DE ORDENACIÓN SUELEN ESTAR SUJETOS A CAMBIOS.



QUEDA EN EL ALUMNO INDAGAR Y PROBAR. ¡SUERTE JUGANDO CON LOS ALGORITMOS!

PD: Cuando lleguen a Algoritmia, me lo agradecerán.